

RabbitMQ

Mensageria em Sistemas Distribuídos

Confiabilidade · Ordenação · Tolerância a Falhas

Arthur Real Sanchotene Ferreira · Caroline da Rocha de Lima · Giovani da Silva Cancherini

Leonardo Vargas Schilling · Osmar Sadi Nether Filho

Sistemas Distribuídos — 2026

Agenda

01

O que é — e por que existe?

02

Arquitetura e Funcionalidades

03

Tipos de Falhas Suportadas

04

Confiabilidade — modelos formais

05

Ordenação de Mensagens

06

Implementação Interna

07

Demonstração Prática — TicketLab

O Problema

HTTP síncrono: uma cadeia que quebra pela parte mais fraca

Imagine o Lollapalooza abrindo vendas às 10h — 50.000 pessoas clicando ao mesmo tempo.

```
Usuário → API → chama Pagamento (aguarda 3s)
              → chama Estoque    (aguarda 1s)
              → chama Email      (aguarda 2s)
              ← responde após 6 segundos

Se Pagamento demorar 30s → Timeout. Usuário vê erro 500.
Se Estoque cair          → Toda a compra falha.
2 usuários simultâneos   → Overbooking possível.
```

► *Dica: abrir no browser antes da apresentação*

[VÍDEO OPCIONAL — SUBSTITUIR]

'RabbitMQ in 100 Seconds' — Fireship (YouTube)

*Buscar: 'RabbitMQ in 100 Seconds Fireship' | Abrir no browser, reproduzir primeiros 45 segundos.
Mostra o problema do HTTP síncrono de forma animada. Pausar antes do slide 4.*

O que falha

- ✗ Timeout em cascata
- ✗ Falha se qualquer serviço cair
- ✗ Overbooking
- ✗ Thread presa até o fim
- ✗ Escalar = problema

A Solução

Desacoplar produção de consumo via broker de mensagens



Resposta imediata

API retorna na hora. Processamento ocorre em background.

Workers independentes

Cada etapa é um serviço separado. Falha num não para os outros.

Escalabilidade natural

Adicionar workers = aumentar throughput sem mudar a API.

O que é o RabbitMQ?

Message broker open-source — o intermediário confiável

Message Broker

Intermediário entre quem envia e quem recebe mensagens.
Desacopla produtor de consumidor no tempo e no espaço.
Não é um banco de dados — é um canal confiável.

Protocolo AMQP

Implementa AMQP 0-9-1 (Advanced Message Queuing Protocol).
Protocolo aberto, binário, com semântica bem definida de ACK.
Também suporta STOMP, MQTT e AMQP 1.0 via plugins.

Erlang / OTP

Escrito em Erlang — linguagem criada para sistemas de telecom com 99,9999999% de disponibilidade (nine nines).
Isolamento de processos e recuperação de falhas nativos.

Onde é utilizado?

Qualquer sistema que precisa desacoplar e escalar

E-commerce

Checkout → pagamento →
estoque → notificação.
Exatamente o nosso
projeto TicketLab.

Sistemas bancários

Processamento assíncrono
de transações.
Auditoria e filas de
conciliação.

IoT e sensores

Ingestão de eventos de
milhares de dispositivos.
Fanout para múltiplos
consumidores analíticos.

Microserviços

Comunicação entre
bounded contexts sem
acoplamento temporal.
Event-driven architecture.

Reservas / Ingressos

Controle de concorrência
em estoque limitado.
Fila garante que só um
worker por vez altera o
estoque.

Arquitetura Interna

Producer → Exchange → Queue → Consumer

[IMAGEM — SUBSTITUIR]

Diagrama oficial — Producer → Exchange → Queue → Consumer

Buscar: 'RabbitMQ Hello World tutorial diagram' em rabbitmq.com/tutorials
Copiar a imagem do tutorial 'Hello World' (fundo branco, setas claras).
Substituir este placeholder. Redimensionar para ocupar o espaço.

Producer

Envia mensagens para o Exchange. Não conhece as filas.

Exchange

Recebe e roteia via Binding + Routing Key. Não armazena.

Queue

Armazena mensagens até o consumer estar pronto.

Consumer

Processa e envia ACK. Só então o broker remove da fila.

Exchange Direct

Como usamos no TicketLab — roteamento pela chave exata

[IMAGEM — SUBSTITUIR]

Direct Exchange — routing_key exata roteia para a fila correspondente

Buscar: 'RabbitMQ direct exchange diagram' em clouddamqp.com/blog/rabbitmq-exchanges-tutorial.html
CloudAMQP tem diagramas limpos e coloridos para cada tipo. Usar o de Direct Exchange.

No TicketLab

```
ticket-sales    → routing_key='payment'    →  
payment-queue  
payment-approved → routing_key='stock'    →  
stock-queue  
stock-confirmed → routing_key='notification' →  
notification-queue
```

Por que Direct?

- ▶ Pipeline linear: cada etapa tem uma fila dedicada
- ▶ Routing simples e previsível
- ▶ Fácil de monitorar no Management UI

Outros Tipos de Exchange

Fanout · Topic · Headers

Fanout

Copia para TODAS as filas vinculadas.
Ignora routing key.
Uso: broadcast de eventos (logs, auditoria, notificações globais).

[IMAGEM — SUBSTITUIR]

Diagrama Fanout Exchange

Buscar: 'RabbitMQ fanout exchange diagram' clouddamqp

Topic

Padrão com wildcards: * (uma palavra) e # (zero ou mais).
'order.#' captura 'order.created', 'order.paid'...
Uso: sistemas de eventos flexíveis.

[IMAGEM — SUBSTITUIR]

Diagrama Topic Exchange

Buscar: 'RabbitMQ topic exchange diagram' clouddamqp

Headers

Roteia por atributos do cabeçalho AMQP.
Mais expressivo que Topic, mais raro na prática.
Uso: roteamento por metadados arbitrários.

[IMAGEM — SUBSTITUIR]

Diagrama Headers Exchange

Buscar: 'RabbitMQ headers exchange diagram' clouddamqp

Funcionalidades

O que o RabbitMQ oferece além do roteamento básico

Mensagens Persistentes

delivery_mode=PERSISTENT → grava em disco antes de confirmar. Sem isso, reinício apaga tudo.

ACK Manual

Consumer confirma só após processar e salvar. Broker só remove ao receber ACK.

QoS / Prefetch

prefetch_count limita msgs em voo por consumer. Backpressure automático.

Filas Durable

Metadados da fila ficam no disco. Broker reinicia → fila existe e mensagens voltam.

Dead Letter Exchange

NACK sem requeue → DLX → Dead Letter Queue para análise e reprocessamento.

Management UI / API

Interface web + HTTP API para monitorar filas, consumers e publicar msgs.

Falha 1: Crash-stop do Worker

O container morre — nenhuma mensagem se perde



Por que não perde mensagens?

- ▶ TCP disconnect detectado
→ RabbitMQ libera msgs unacked
- ▶ Outras msgs na fila:
estado ready, esperando
- ▶ Worker B (ou C) pega
as mensagens liberadas
- ▶ Sem intervenção manual —
automático

```
make send-1000      # enche a fila
make kill-payment    # mata um worker abruptamente
make logs-payment    # ver outro worker assumindo as mensagens
```

[GIF / SCREENSHOT — SUBSTITUIR]

Terminal: make kill-payment + make logs-payment

*Gravar GIF do terminal com ScreenToGif (Windows) ou OBS
executando os 3 comandos acima.*

Alt: screenshot de logs mostrando outro worker retomando msgs.

Falha 2: Crash-recovery + Retry + DLQ

O broker reinicia · A lógica falha · O destino final

Crash-recovery do Broker

- ▶ RabbitMQ reinicia (docker restart)
- ▶ Mensagens PERSISTENT + filas durable → no disco
- ▶ Workers reconectam automaticamente (connect_robust)
- ▶ Sistema retoma de onde parou

```
make restart-rabbit
```

[SCREENSHOT — SUBSTITUIR]

RabbitMQ UI após restart — filas com mensagens intactas

Retry com Dead Letter Queue

```
msg → worker → lógica falha (RuntimeError)
  → republica com x-retry-count: 1 → ACK original
  → falha de novo → x-retry-count: 2
  → falha de novo → x-retry-count: 3
  → max_retries atingido!
  → NACK(requeue=False) → dead-letter-exchange → dead-letter-queue
```

Depois do make send-failures:

[SCREENSHOT — SUBSTITUIR]

RabbitMQ UI — dead-letter-queue com N mensagens

Modelos de Falha — Resumo

O que suportamos vs. o que não suportamos

✓ Suportado

✓ Crash-stop

Worker morre → mensagens voltam para a fila automaticamente

✓ Crash-recovery

Broker reinicia → mensagens PERSISTENT sobrevivem no disco

✓ Omissão com retry

Falhas de lógica → DLQ após N tentativas

✓ Partição de rede

connect_robust reconecta com backoff exponencial

✗ Não suportado

✗ Falhas Bizantinas

Nó malicioso enviando mensagens forjadas ou corrompidas

✗ Exatamente-uma-vez nativo

At-least-once → duplicatas possíveis (tratamos com idempotência)

Falhas Bizantinas requerem protocolo BFT dedicado (PBFT, Tendermint). Fora do escopo do RabbitMQ.

Confiabilidade: Fair-Loss vs Perfect Link

O que acontece com as mensagens quando o broker cai?

Fair-Loss Link

Msgs podem se perder, mas não são inventadas.

Configuração:

```
exchange → non-durable
fila      → non-durable
delivery  → NON_PERSISTENT (só RAM)
```

Resultado:

- ▶ Broker reinicia → msgs na RAM somem
- ▶ Nenhum aviso ao producer
- ▶ Garantia? Nenhuma.

Perfect Link (at-least-once)

Toda msg enviada é eventualmente entregue.

Nossa configuração:

```
exchange → durable
fila      → durable
delivery  → PERSISTENT (disco)
ACK       → manual (só após processar)
conexão   → connect_robust (reconecta)
```

Ressalva: at-least-once

Worker processa + falha antes do ACK → recebe de novo.
Idempotência (próximos slides) trata isso.

BEB vs Reliable Broadcast

O produtor falha no meio — o que acontece?

Best Effort Broadcast

O emissor dispara e não garante se ele mesmo falhar.

No projeto — endpoint POST /batch:

```
# publisher.py — sem publisher confirms
await exchange.publish(msg, routing_key='payment')
# Se broker cair AQUI → msg perdida. Producer não sabe.
```

Analogia: UDP em broadcast.

- ▶ Enviamos 1000 msgs
- ▶ Broker cai na msg 500
- ▶ Msgs 501-1000 perdidas
- ▶ Producer não tem como saber

Reliable Broadcast (aprox.)

Se um processo correto entrega m, todos entregam m.

Nossa aproximação:

- ▶ Msg que chega ao broker com PERSISTENT → nunca some
- ▶ ACK manual → broker guarda até worker confirmar
- ▶ Worker ativo vai processar eventualmente

O que ainda falta para RB formal:

- ▶ Publisher confirms (sem eles = BEB)
- ▶ Se producer cai antes de publicar, não há como recuperar

Uniform Reliable Broadcast

O nível mais alto — e como chegamos perto

Definição de URB:

Se qualquer processo (correto OU com falha) entrega $m \rightarrow$ TODOS os processos entregam m . Requer consenso.

✗ RabbitMQ não garante URB nativo

Cenário problemático:

```
Worker A: recebe msg → processa parcialmente  
          → salva metade no banco  
          → FALHA antes do ACK
```

```
Worker B: recebe a mesma msg de novo  
          → estado parcial do Worker A já existe  
          → sem consenso sobre quem entregou
```

✓ Nossa solução: idempotência

Verificamos o estado antes de processar:

```
# notification-worker/worker.py  
if order.status == 'confirmed':  
    return # já processado, ignora duplicata  
  
# Garante: efeito visível = exactly-once  
# mesmo com at-least-once na fila
```

Resultado: exactly-once semântico para o usuário.

Comparativo — os 5 Modelos

De Fair-Loss até URB — onde o RabbitMQ se encaixa

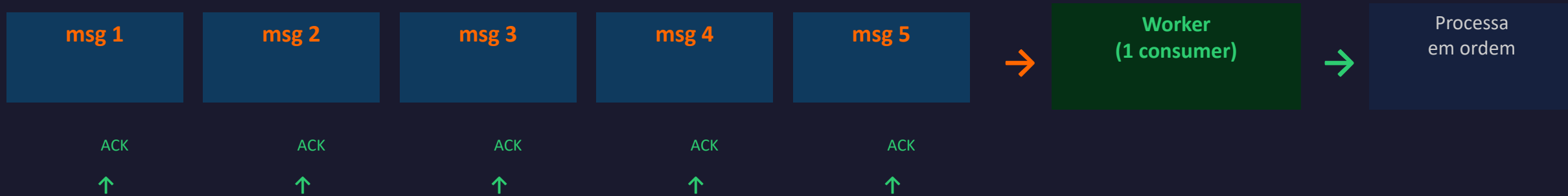
Modelo	Perde msg?	Duplicatas?	Crash producer	No projeto
Fair-Loss Link	Sim	Não	—	Sem persistência
Perfect Link	Não	Possível	Perde msgs	PERSISTENT + ACK + durable
BEB	Possível	Não	Perde msgs	POST /batch (sem confirms)
Reliable Broadcast	Não	Possível	Perde msgs	Aprox. com persistência
Uniform RB	Não	Não	Nenhuma	Não nativo (idempotência)

Linha destacada em laranja = nossa implementação principal.

Ordenação — FIFO com 1 Consumer

A fila garante a ordem. Um consumidor respeitá-la.

Fila (payment-queue)



Propriedade FIFO garantida pelo RabbitMQ:

A especificação AMQP 0-9-1 define que mensagens publicadas com a mesma routing key e priority chegam à fila na ordem em que foram publicadas — e são entregues nessa ordem a um único consumer.

Mas: com N consumers em paralelo, a ordem global de processamento NÃO é garantida. (Slide 19)

[GIF OPCIONAL — SUBSTITUIR]

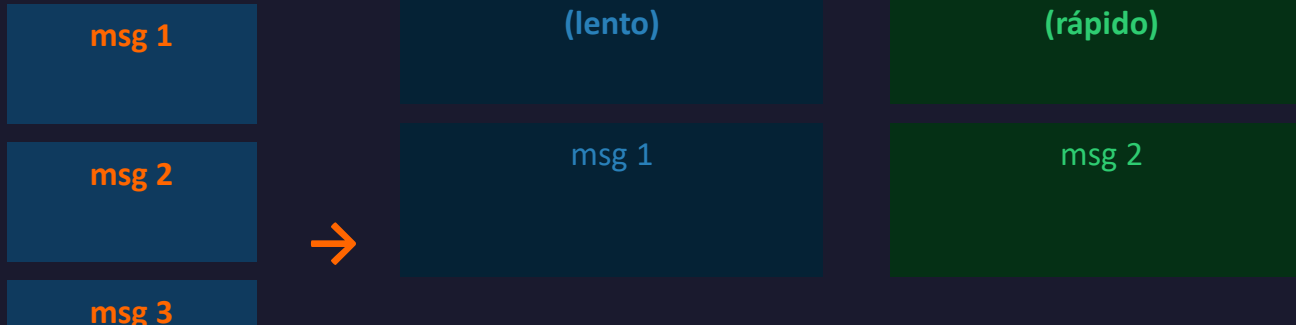
Animação de fila FIFO

Buscar: 'queue FIFO animation gif' ou 'message queue gif'.
Visualfsm.js e CloudAMQP têm animações boas.

Ordenação — N Consumers

Paralelo quebra a ordem global. Como resolver.

Fila



```
make scale-workers N=5 && make send-100  
make db-query  
# processed_at NÃO respeita created_at
```

Resultado:

Worker B termina primeiro: msg 2 confirmada antes de msg 1.
Ordem de chegada: 1,2,3,4
Ordem de processamento: 2,4,1,3
→ Ordem global quebrada

Como garantir ordem total — se necessário:

► Particionamento

1 worker por event_id (como Kafka faz com partition keys). Simples e eficaz.

► Sequencer

Nó central atribui número de sequência antes de publicar. Gargalo, mas garante ordem.

► Kafka

Nativo: cada partição é uma fila FIFO. Workers de uma partição respeitam a ordem.

Implementação: Persistência + ACK

Como o RabbitMQ garante que mensagens não somem

1	Publisher	<code>publish(PERSISTENT)</code>	Grava em disco antes de retornar
2	Disco	mnesia / msg store	Arquivo em /var/lib/rabbitmq
3	Consumer	<code>processa()</code>	Lógica de negócio + commit no banco
4	ACK	<code>channel.ack()</code>	Broker remove da fila (disco)
!	Sem ACK	crash antes do ack	Msg volta para ready → at-least-once

At-least-once: se o worker processa e cai ANTES do ACK, recebe a mesma mensagem novamente. Necessário: idempotência no processamento.

Implementação: Anti-Overbooking

SELECT FOR UPDATE previne a venda dupla do mesmo ingresso

Sem controle — o que pode acontecer:

```
Worker A lê: available_tickets = 1 → reserva  
Worker B lê: available_tickets = 1 → reserva (ao mesmo tempo!)  
  
Resultado: 2 vendas, 0 ingressos → OVERBOOKING
```

Com SELECT FOR UPDATE — nossa solução:

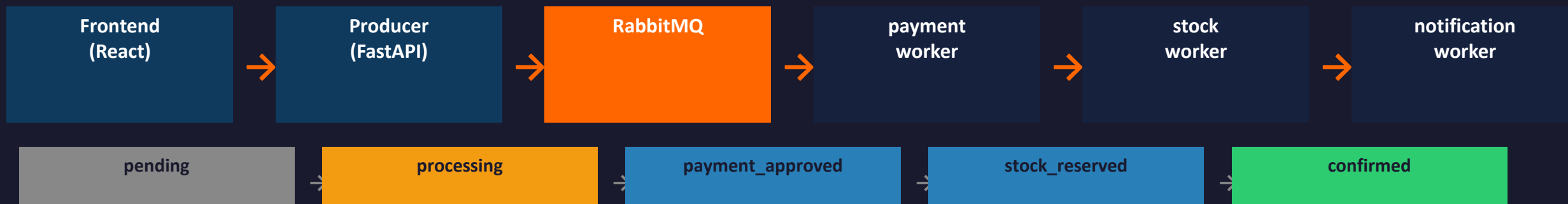
```
-- stock-worker/src/database.py → reserve_tickets()  
BEGIN;  
  SELECT * FROM events  
  WHERE id = $1  
  FOR UPDATE;          -- bloqueia outros workers aqui  
  
  UPDATE events SET available_tickets -= $qty;  
  UPDATE orders SET status = 'stock_reserved';  
COMMIT;                -- lock liberado, próximo worker entra
```

O que acontece:

- ▶ Worker B tenta e bloqueia
- ▶ Aguarda Worker A commitar
- ▶ Se estoque zerou: recebe out_of_stock
- ▶ Sem overbooking possível

TicketLab — Sistema de Venda de Ingressos

Materializa todos os conceitos vistos — RabbitMQ não é infraestrutura oculta, é o protagonista.



Conceitos demonstrados:

- ▶ Perfect Links → filas durable + PERSISTENT + ACK manual
- ▶ Crash-stop → make kill-payment — sem perda de mensagens
- ▶ Race condition → SELECT FOR UPDATE — zero overbooking
- ▶ DLQ → make send-failures (30% falha) → make inspect-dlq
- ▶ Escalabilidade → make scale-workers N=5

[SCREENSHOT — SUBSTITUIR]

Frontend TicketLab — tela principal com eventos

Demo: Saga Coreografada

Um pedido passa por 4 etapas — cada uma é um worker independente

1. Compra

Formulário preenchido → POST /orders
API retorna { status: 'pending' } imediato.
Publica em ticket-sales (routing_key='payment').

[SCREENSHOT — SUBSTITUIR]

SCREENSHOT: tela de compra preenchida — localhost:3000/buy
Capturar com o formulário preenchido antes de clicar.

2. Pagamento

payment-worker consome payment-queue.
Valida → publica em payment-approved.
Status: 'processing'.

[SCREENSHOT — SUBSTITUIR]

SCREENSHOT: painel Orders mostrando status 'processing'
localhost:3000/orders — filtrar por status.

3. Estoque

stock-worker consome stock-queue.
SELECT FOR UPDATE → reserva ingresso.
Status: 'stock_reserved'.

[SCREENSHOT — SUBSTITUIR]

(capturar em sequência, mesmo vídeo se possível)

4. Confirmação

notification-worker gera ticket_code UUID.
Verifica idempotência → status: 'confirmed'.
Ingresso na mão do comprador.

[SCREENSHOT — SUBSTITUIR]

SCREENSHOT: pedido com status 'confirmed' e ticket_code
localhost:3000/orders → detalhe do pedido.

Demo: Falhas na Prática

Três cenários — o sistema é resiliente

Dead Letter Queue

```
make send-failures  
→ 30% de pedidos com  
simulate_failure  
→ 3 retries → NACK → DLQ
```

```
make inspect-dlq
```

[SCREENSHOT — SUBSTITUIR]

SCREENSHOT: RabbitMQ UI
Queues → dead-letter-queue
N mensagens acumuladas
localhost:15672

Crash-stop Worker

```
make send-1000  
make kill-payment  
make logs-payment
```

```
→ outro worker assume
```

[SCREENSHOT — SUBSTITUIR]

SCREENSHOT / GIF: Grafana
consumer count caindo de 2→1
e fila sendo drenada
localhost:3001

Crash-recovery Broker

```
make restart-rabbit  
→ msgs PERSISTENT no disco  
→ workers reconectam
```

```
connect_robust (backoff)
```

[SCREENSHOT — SUBSTITUIR]

SCREENSHOT: RabbitMQ UI
após restart — filas intactas
localhost:15672 → Queues

Conclusão

O que aprendemos sobre o RabbitMQ em Sistemas Distribuídos

Questão do Professor	Resposta
Onde é usado?	E-commerce, IoT, bancos, microserviços — desacoplamento temporal
Funcionalidades?	Exchanges, durable queues, PERSISTENT, ACK, DLX, QoS, plugins
Falhas suportadas?	Crash-stop, crash-recovery, retry+DLQ. Não suporta Byzantine.
Confiabilidade?	Perfect Links (at-least-once) + aprox. Reliable Broadcast
Ordenação?	FIFO com 1 consumer. N consumers → requer particionamento.
Implementação? Próximos passos para produção:	AMQP 0-9-1, Erlang/Mnesia, protocolo ACK, SELECT FOR UPDATE

- ▶ Publisher confirms → Reliable Broadcast formal (saíndo do BEB)
- ▶ Outbox Pattern → zero perda durante partição producer↔broker
- ▶ Quorum Queues (Raft) → aproximação de URB nativa no RabbitMQ 3.8+
- ▶ Kafka + partition keys → ordem total por evento

Obrigado. Ficamos à disposição para perguntas.